Data Lab 实验报告

姓名：王昊昇
学号：24307130030

```
hswang@localhost:~/ICS26-lab1-datalab-hswang$ ./check_ops.py bits.c
signMask: 1/2 operators
bitXor: 7/8 operators
negativePart: 4/6 operators
copyByteWithin: 9/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 9/18 operators
secondLowestZeroBit: 8/8 operators
oddParity: 12/56 operators
rotateRightBits: 12/16 operators
roundEvenPow2: 11/24 operators
midpointTowardFirst: 25/32 operators
isBetweenEitherOrder: 38/48 operators
mul5Sat: 21/30 operators
classifyAdd3: 31/52 operators
floatScaleThreeHalves: 34/60 operators
floatRoundEven: 24/65 operators
float_i2f: 32/40 operators
bitCount: 28/40 operators
bitReverse: 34/34 operators
All 19 functions passed operator checks.
```

```
hswang@localhost:~/ICS26-lab1-datalab-hswang$ ./btest
Score   Rating  Errors  Function
 1      1       0       signMask
 2      2       0       bitXor
 3      3       0       negativePart
 4      4       0       copyByteWithin
 4      4       0       logicalShift
 4      4       0       swapNibblePairs
 4      4       0       secondLowestZeroBit
 5      5       0       oddParity
 5      5       0       rotateRightBits
 5      5       0       roundEvenPow2
 5      5       0       midpointTowardFirst
 7      7       0       isBetweenEitherOrder
 7      7       0       mul5Sat
 7      7       0       classifyAdd3
 7      7       0       floatScaleThreeHalves
 10     10      0       floatRoundEven
 10     10      0       float_i2f
 10     10      0       bitCount
 10     10      0       bitReverse
Total points: 110/110
```

各题实现思路

P1

将整数1左移31位，就得到只有最高位为1的掩码。

P2

异或要求两位不同时结果为1，用`~(x&y)`排除两位都为1的情况，用`~(~x&~y)`排除两位都为0的情况，再将两者按位与。

P3

用x>>31生成符号掩码，再与~x+1相与，使负数返回其相反数，非负数返回0。INT_MIN按实验的32位补码规则保留低32位，结果仍为INT_MIN。

P4

先通过右移和255提取源字节，再用掩码清空目标字节，把源字节移到目标位置，用按位或合并。

P5

先对x做算术右移，再用低32-n位为1的掩码清除高位的符号扩展。掩码的构造也能处理n=0和n=31，不会出现移动32位的情况。

P6

构造`0x0f0f0f0f`掩码，分别提取每个字节的低4位和高4位，交换位置后合并。

P7

x取反，将寻找0位转化为寻找1位，再利用v&(~v+1)提取最低的1位，清除第一个1位后再次提取，就得到原数第二低的0位对应的掩码，不足两个0位时结果为0。

P8

依次将右移16、8、4、2、1位的结果与原值异或，把所有位的奇偶信息合并到最低位。最后对最低位取逻辑非，使1的个数为偶数时返回1

P9

先用n&31把移动位数限制到0至31，因为循环移动32位相当于没有移动。结果由两部分组成：逻辑右移， 把被移出的低位放到高位，最后按位或合并。左移位数也按32取模，避免右移位数为0时出现左移32位的情况，两部分合并后仍为原数。

P10

把x看成整数商和余数两部分，其中商为`x>>n`，半个步长为half。给x加上half-1，商为奇数时再多加1，然后右移`n`位得到舍入后的商。这样余数小于半程时不进位，大于半程时进位，恰好等于半程时则由商的奇偶性决定。原商为偶数就保留，为奇数就加1，从而实现中点取偶，例如10取最近的4的倍数得到8，14则得到16。最后左移`n`位，把商还原为对应的倍数。

P11

直接计算`x+y`可能溢出，因此先用(x&y)+((x^y)>>1)得到向下取整的中点。这个式子把两数共同为1的部分和不同的部分分开处理，避免先生成完整的和。只有两数奇偶性不同时，中点才会落在两个整数之间，此时可通过`(x^y)&1`判断。若`x>y`，向下取整的结果离`x`较远，需要加1；否则保持原结果。比较大小时，异号直接看符号，同号再看差值，避免减法溢出影响判断。

P12

先分别判断x>=a和x>=b，每次比较都区分同号与异号：异号看符号，同号看差值。反向的a>=x和b>=x可以由前面的结果再加上相等判断得到，不必重复计算完整的比较。然后组合出a<=x<=b或b<=x<=a，这样无论哪个端点更大都能处理，也包含恰好等于端点的情况。最后用两次逻辑非把结果统一为0或1。

P13

先计算`four=x<<2`，再加上`x`得到乘5的低32位结果。溢出可能发生在左移乘4或后续加法中，因此需要分别检查。若`four>>2`不能恢复原来的`x`，说明乘4时已经丢失了有效信息。若乘4没有溢出，`four`与`x`同号，此时乘5结果与`x`异号就说明加法溢出。把两个判断合并成掩码后，根据原数符号选择`INT_MAX`或`INT_MIN`，没有溢出则保留乘积。

P14

把三数相加拆成`x+y`和再加z两步，分别记录每次加法的溢出方向。两个非负数相加却得到负数时记一次正溢出，两个负数相加却得到非负数时记一次负溢出。正溢出意味着精确结果比当前有符号结果多一个2^32 负溢出则意味着少一个2^32。因此，正溢出次数减去负溢出次数就能表示最终结果是否超出有符号整数范围。两步的溢出可能相互抵消，例如

INT_MAX+INT_MAX+INT_MIN的精确结果仍在范围内，中间两次加法却分别发生正溢出和负溢出。实现中将两步记录合并，得到1表示正向越界，-1表示负向越界，0表示没有越界。

P15

先拆出符号位、阶码和尾数，遇到NaN或无穷大直接返回原编码。对规格化数补上隐含的最高位1，非规格化数则直接使用尾数字段，再通过移位和加法将有效数乘3。乘3后通常右移1位来完成除2；如果结果需要提高阶码才能保持规格化，则右移2位，并将阶码加1。右移时保留被丢弃的部分，将它与半程值比较：超过半程就进位，恰好等于半程时仅在保留部分为奇数时进位。这样只在确定最终移位量后舍入一次，避免连续舍入造成误差。若舍入使有效数再多出一位，就继续调整有效数和阶码。非规格化数的结果达到规格化范围时要设置阶码，阶码上溢时返回对应符号的无穷大，而正负零始终保留原符号。

P16

先根据阶码判断范围，NaN和无穷大原样返回，阶码达到150的有限数也不需要处理，因为这种单精度数已经没有小数部分。绝对值小于0.5时直接返回带原符号的零。绝对值在0.5到1之间时，恰好等于0.5取零，其余取带符号的1。剩下的情况用150-exp确定需要去掉的低位数，再用掩码分离小数部分，并先把这些位清零。将小数部分与半程值比较，超过半程时进位，恰好处于半程时根据整数部分的奇偶性决定。进位是在代表整数最低位的位置加1，因而得到的仍是浮点数编码，例如1.5变成2.0，2.5也变成2.0。负数使用同样的绝对值舍入规则，符号位保留，舍入为零时也能保持负零。

P17

先单独处理0，再记录原数符号，并按实验的补码规则把绝对值位模式存入unsigned变量。使用无符号数可以表示`INT_MIN`对应的大小2^31。从高位向低位找到最高的1，若它位于第p位，则浮点数的阶码为p+127。当有效位不超过24位时，直接左移到正确位置，转换不会损失精度。超过24位时保留高24位，并记录丢弃部分，与半程值比较后进行最近偶数舍入。若舍入使保留部分增加到25位，就再右移一位，同时增加阶码。最后去掉隐含的最高位1，将尾数字段与阶码、符号位组合起来。

P18

采用分组累加的方法，先用0x55555555掩码分别取出相邻位并相加，得到每2位中的1的个数。再用0x33333333合并成每4位的计数，随后通过移位、相加和0x0f0f0f0f掩码得到每个字节的计数。此时每个字节最多计到8，能够继续用右移8位、16位和相加把计数汇总到低位。高位留下的中间结果不影响最终低位计数，总数最多为32，因此最后取低6位即可。

P19

采用逐层交换的方法，先交换相邻的两个位，再交换相邻的2位分组，之后处理4位、8位和16位分组。每完成一层，更大的分组内部就已经按相反顺序排列，最后得到整个32位数的反转。交换时分别提取左右两部分，向相反方向移动后合并，右移后的掩码同时负责清除符号扩展。为了满足34次操作的限制， 先生成0x0000ffff，再通过移位和异或逐步得到字节、半字节和更小分组的掩码。最后交换高低16位时，左移会自然丢弃原高位，因此这一侧不必再加掩码。

参考资料与工具

- 2026 DataLab实验说明
- cmu ics课程项目
- 25年ICS课程Lab1
- AI使用情况说明：本次6、7、9-19题解题思路、调试和报告草稿整理使用了AI辅助